Week 2 – Day 1

Home Assignment :

1. Prepare a test strategy document based on the domain, using a chain-of-thought approach (refer to the document provided in the ZIP file)

2. Design the automation framework structure using a chain-of-thought approach

Output: Analyze and review the generated results

Task 1: Tests Strategy document based on domain w/ chain-of-thought-prompting

---

Prompt

Act as a Senior Test Automation Architect with deep expertise in fintech/payment systems.

Create a comprehensive Test Strategy Document for a Payment Gateway system using a structured, risk-based approach. The strategy should be suitable for API-first systems and aligned with pre-deployment smoke testing and regression needs.

Context:
The payment system supports **[Critical]**:

- UPI transfers
- Wallet deposits
- Wallet withdrawals
- Account balance queries
- Transaction history with date filters (from/to date inputs)
- Configurable rules such as:
    - Cap on fund transfers without PAN
    - Cap on deposits without PAN
    - Daily withdrawal limits

Expectations **[Mandatory]**:

1. Structure the document professionally with clear sections such as:
    - Objective
    - Scope (in-scope and out-of-scope)
    - Risk Analysis (P0/P1/P2 classification)
    - Testing Levels (Smoke, Functional, Integration, Regression, Non-functional)
    - Test Design Techniques
    - Test Data Strategy (include boundary values)
    - Automation Strategy (API-first using Playwright or equivalent)
    - Idempotency and consistency validation approach

    - Observability and monitoring validation
    - Entry and Exit Criteria
    - Domain-specific considerations for payments
2. Focus strongly on payment-domain risks:
    - Double debit/credit
    - Idempotency failures
    - Ledger vs wallet inconsistencies
    - Regulatory violations (PAN/KYC limits)
    - Concurrency issues
3. Include:
    - Boundary condition examples (e.g., ₹50,000 vs ₹50,001)
    - Critical assertions for financial correctness
    - Data isolation strategy for repeatable tests
    - CI/CD integration approach
4. The output should:
    - Be concise but senior-level in depth
    - Avoid generic testing theory—prioritize practical, real-world strategies
    - Be directly usable as a professional test strategy document (e.g., Confluence-ready)
5. Do NOT include step-by-step reasoning or chain-of-thought. Provide only the final structured strategy.

Tone:

- Authoritative and practical
- Focused on risk mitigation and system reliability
- Tailored for high-stakes financial systems

Output Format:

- Clean headings and bullet points
- Tables where useful (e.g., risk classification, test coverage)
- No fluff or unnecessary explanations

Output:

🧾 Payment Gateway Test Strategy (API + System Level)

1. 🎯 Objective

Establish a **risk-driven, automation-first testing strategy** to ensure:

- Transaction integrity (no loss/duplication of money)
- Regulatory compliance (KYC/PAN limits)
- High availability & fault tolerance
- Safe and fast pre-deployment regression (smoke coverage)

---

2. 🧩 System Scope

Core Functional Areas

- UPI Transfers
- Wallet Deposits
- Wallet Withdrawals
- Balance Management
- Transaction History & Filtering
- Configurable Limits (PAN thresholds, daily caps)

External Dependencies

- Banking/UPI PSP systems
- KYC/PAN validation services
- Ledger/Accounting system
- Notification systems (SMS/email)

---

3. ⚠️ Risk-Based Testing Focus

Critical Risks (P0)

- 💸 Double debit / credit
- 🔁 Idempotency failures
- ⚖️ Ledger mismatch vs wallet balance
- ⛔ Limit enforcement bypass (PAN rules)
- 🔐 Unauthorized transactions

High Risks (P1)

- Inconsistent transaction states (PENDING → FAILED)
- Date filter inaccuracies in transaction history
- Race conditions in concurrent withdrawals

Medium Risks (P2)

- Reporting delays
- Notification failures

---

4. 🧪 Testing Levels & Strategy

A. Smoke Testing (Pre-Deployment Gate)

- Minimal set of **API-level Playwright tests**
- Covers:
  - UPI transfer success/failure
  - Deposit/withdrawal limits
  - Balance correctness
  - Transaction retrieval
- Execution time target: **< 5 minutes**

B. Functional Testing

- Positive + negative flows
- Boundary testing (limits, min/max amounts)
- Config-driven validations (dynamic caps)

C. Integration Testing

- Validate:
  - PSP responses (success/failure/timeout)
  - Ledger sync
- Use:
  - Service virtualization (mock PSP)

D. Regression Testing

- Triggered on:
  - Any payment-related code change
- Includes:
  - Full API suite
  - Data consistency checks

E. Non-Functional Testing

Performance

- Peak load: concurrent transactions
- SLA: < 2 sec response time

Reliability

- Retry mechanisms
- Circuit breaker validation

Security

- Auth token validation
- Injection attacks
- Replay attack prevention

---

5. 🔄 Test Design Approach

Techniques Used

- Boundary Value Analysis (e.g., ₹50,000 vs ₹50,001)
- Equivalence Partitioning
- State Transition Testing (PENDING → SUCCESS/FAILED)
- Pairwise testing (limits + user types)

---

6. 📊 Data Strategy

Test Data Types

- PAN verified vs non-verified users
- Low/medium/high balance accounts
- Edge cases:
    - Zero balance
    - Max limit transactions
    - Duplicate request IDs

Data Isolation

- Use **unique user IDs per test run**
- Reset state via:
    - Test hooks / seeded DB

---

7. 🤖 Automation Strategy

Tooling

- API Automation: Playwright
- CI/CD: Jenkins / GitHub Actions

Layers

- API test layer (primary)
- Contract validation layer
- Mock layer (PSP simulation)

Execution Plan

Stage	Tests
Pre-commit	Unit tests
Pre-deploy	Smoke tests
Nightly	Full regression
Release	Performance + security

---

8. 📌 Key Assertions (Must-Have)

- Transaction status correctness
- Balance delta validation
- No duplicate transaction IDs
- Limit enforcement (PAN rules)
- API response schema validation

---

9. 🔁 Idempotency & Consistency Strategy

- Validate:
  - Same `requestId` → no duplicate debit
- Ensure:
  - Eventual consistency between:
    - Wallet
    - Ledger
    - Transaction logs

---

10. 📉 Observability & Monitoring

- Validate logs:
  - Transaction lifecycle
- Metrics:
  - Success rate
  - Failure rate
  - Retry count
- Alerts:
  - Spike in failures
  - Latency breaches

---

11. 🚫 Entry & Exit Criteria

Entry

- APIs deployed in staging
- Test data ready
- Dependencies mocked/available

Exit

- 100% smoke pass
- No P0/P1 defects
- Balance reconciliation validated

---

12. 🧠 Domain-Specific Considerations (Payments)

- **Money is stateful → every test must verify balance**
- **Idempotency is mandatory, not optional**
- **Auditability is critical (every transaction traceable)**
- **Regulatory rules (PAN/KYC) must be enforced strictly**
- **Failure ≠ rollback always (must handle partial states)**

---

13. 📌 Example Traceability Mapping

Feature	Risk	Test Type
UPI Transfer	Double debit	Smoke + Regression
Deposit	Limit breach	Boundary tests
Withdrawal	Overdraw	Negative tests

Feature	Risk	Test Type
Transactions	Incorrect filters	Functional
Config limits	Misconfiguration	API validation

✅ Final Takeaways

- Focus on **money safety > feature correctness**
- Prioritize **idempotency + consistency checks**
- Keep **smoke suite lean but high-impact**
- Automate **boundary + limit validations aggressively**
- Treat **every transaction as auditable evidence**

If you want, I can next:

- Convert this into a **formal PDF/Confluence-ready doc**
- Map this strategy to your **existing Playwright suite**
- Add a **risk-based prioritization matrix with scoring**

End of Task 1

Task 2: Design the automation framework structure using a chain-of-thought approach

---

Prompt (I made it very detailed and specific)

Act as a Senior Test Automation Architect specializing in fintech/payment systems.

Design a robust, scalable, and production-grade API automation framework structure using Playwright (or equivalent), based on a previously defined payment gateway test strategy.

Context:
The system under test includes:

- UPI transfers
- Wallet deposits

- Wallet withdrawals
- Balance queries
- Transaction history with date filters
- Configurable limits:
    - Transfer cap without PAN
    - Deposit cap without PAN
    - Daily withdrawal limit

Objective:
Design an automation framework that supports:

- Pre-deployment smoke testing
- Full regression testing
- High reliability for financial transactions

Expectations:

1. Framework Architecture
    - Define folder structure (tests, utils, services, fixtures, config, data, mocks)
    - Explain separation of concerns (test vs service vs validation layers)
    - Support API-first testing with extensibility for UI if needed
2. Core Components
    - API client layer (request handling, auth, retries, headers)
    - Service layer (UPI, wallet, transactions, config APIs)
    - Test layer (smoke, regression, negative, boundary)
    - Validation/assertion utilities (financial assertions, schema validation)
    - Test data management module (dynamic + boundary data)
    - Idempotency handling utilities
3. Domain-Specific Design
    - Balance reconciliation utilities
    - Transaction state validation (PENDING → SUCCESS/FAILED)
    - Duplicate request protection (idempotency keys)
    - Limit validation helpers (PAN rules)
4. Test Execution Strategy
    - Tagging (smoke, regression, critical)
    - Parallel execution strategy with data isolation
    - Environment configuration (dev/stage/prod-like)
    - Retry and flakiness handling
5. CI/CD Integration
    - How the framework integrates into pipelines
    - Pre-deployment smoke gate design
    - Reporting (logs, HTML reports, failure diagnostics)
6. Observability & Debugging
    - Logging strategy (request/response, correlation IDs)
    - Failure artifacts (API traces, payload snapshots)
7. Non-Functional Support

    - Hooks for performance testing
    - Security test extensibility
8. Example Implementation Snippets
    - Sample folder structure (tree format)
    - Example API client
    - Example service class (e.g., UPI service)
    - Example test (smoke test)
    - Example custom assertion (balance validation)

Constraints:

- Focus on real-world, production-grade design (not academic)
- Prioritize reliability, maintainability, and scalability
- Reflect payment-domain risks (idempotency, consistency, limits)
- Keep the framework modular and reusable

Important:

- Do NOT include step-by-step chain-of-thought reasoning
- Provide only the final structured design with explanations

Tone:

- Senior, practical, and implementation-focused
- Avoid generic statements—be specific to payments and API automation

Output Format:

- Clear sections with headings
- Code snippets where relevant
- Concise explanations with strong rationale

OUTPUT:

🧾 1. Framework Architecture

📁 Folder Structure (Scalable + Modular)

```
payment-automation-framework/
│
├── config/
│   ├── env.ts
│   ├── playwright.config.ts
│
├── tests/
│   ├── smoke/
│   ├── regression/
│   ├── negative/
│   ├── boundary/
│
├── services/
│   ├── upi.service.ts
│   ├── wallet.service.ts
│   ├── transaction.service.ts
│   ├── config.service.ts
│
├── client/
│   ├── apiClient.ts
│
├── fixtures/
│   ├── testFixtures.ts
│
├── utils/
│   ├── assertions.ts
│   ├── validators.ts
│   ├── logger.ts
│   ├── idempotency.ts
│
├── data/
│   ├── testData.ts
│   ├── boundaryData.ts
│
├── mocks/
│   ├── psp.mock.ts
│
├── reports/
│
└── package.json
```

🧠 Separation of Concerns

Layer	Responsibility
Tests	Business scenarios (readable, minimal logic)
Services	API abstraction (UPI, wallet, etc.)

Layer	Responsibility
Client	HTTP handling (auth, retries)
Utils	Assertions, validation, logging
Data	Static + dynamic test inputs
Fixtures	Test setup/teardown

👉 Ensures:

- Maintainability
- Reusability
- Easy debugging

⚙️ 2. Core Components

🔌 API Client Layer

```
// client/apiClient.ts
import { request, APIRequestContext } from '@playwright/test';

export class APIClient {
  private context: APIRequestContext;

  async init(baseURL: string, token: string) {
    this.context = await request.newContext({
      baseURL,
      extraHTTPHeaders: {
        Authorization: `Bearer ${token}`,
        'Content-Type': 'application/json'
      }
    });
  }

  async post(url: string, data: any) {
    const response = await this.context.post(url, { data });
    return this.handleResponse(response);
  }

  async get(url: string) {
    const response = await this.context.get(url);
    return this.handleResponse(response);
  }

  private async handleResponse(res) {
    const body = await res.json();
    return { status: res.status(), body };
  }
}
```

✅ Centralizes:

- Auth
- Headers
- Retry hooks (extendable)
- Error handling

🧩 Service Layer (Example: UPI)

```
// services/upi.service.ts
export class UPIService {
  constructor(private client) {}

  async transfer(payload) {
    return this.client.post('/upi/transfer', payload);
  }
}
```

✔ Keeps tests clean and domain-driven

🧪 Test Layer

- Smoke → critical flows only
- Regression → full coverage
- Boundary → limits validation
- Negative → error scenarios

✅ Assertion Utilities

```
// utils/assertions.ts
export function assertBalanceChange(before, after, delta) {
  if (after !== before + delta) {
    throw new Error(`Balance mismatch: expected ${before + delta}, got
${after}`);
  }
}
```

🧾 Test Data Module

```
// data/boundaryData.ts
export const LIMITS = {
  TRANSFER_NO_PAN: 50000,
  DEPOSIT_NO_PAN: 200000,
  WITHDRAWAL_DAILY: 25000
};
```

🔁 Idempotency Utility

```
// utils/idempotency.ts
export function generateRequestId() {
  return `req-${Date.now()}-${Math.random()}`;
}
```

💳 3. Domain-Specific Design

💰 Balance Reconciliation

```
export async function validateBalanceConsistency(client, expectedDelta) {
  const before = await client.get('/wallet/balance');
  // perform transaction
  const after = await client.get('/wallet/balance');

  assertBalanceChange(before.body.balance, after.body.balance,
expectedDelta);
}
```

🔄 Transaction State Validation

- Validate lifecycle:
  - PENDING → SUCCESS
  - PENDING → FAILED
- Poll API until final state

🔐 Idempotency Enforcement

- Same `requestId` → single transaction
- Assert:
  - No duplicate debit
  - Response = DUPLICATE

📏 Limit Validation Helpers

```
export function isWithinLimit(amount, limit) {
  return amount <= limit;
}
```

🚀 4. Test Execution Strategy

🏷️ Tagging

```
test.describe.configure({ tag: ['@smoke'] });
```

Tag	Purpose
@smoke	Pre-deploy sanity
@regression	Full suite
@critical	High-risk

⚡ Parallel Execution

- Use **isolated users per test**
- Avoid shared wallet state

🌍 Environment Config

```
export const ENV = {
  baseURL: process.env.BASE_URL,
  token: process.env.TOKEN
};
```

🔁 Retry Strategy

- Retry only **network/5xx failures**
- Never retry financial assertions blindly

🔄 5. CI/CD Integration

Pipeline Flow

1. Build
2. Deploy to staging
3. Run **@smoke tests**
4. Block release if failure

Reporting

- HTML reports (Playwright default)
- Attach:
  - Request/response logs
  - Failure snapshots

🔍 6. Observability & Debugging

📜 Logging

```
// utils/logger.ts
export function logRequest(url, payload) {
  console.log(`REQ: ${url}`, payload);
}
```

✅ Include:

- Correlation IDs
- Transaction IDs

📦 Failure Artifacts

- API traces
- JSON payload snapshots
- Logs per test

📈 7. Non-Functional Support

⚡ Performance Hooks

- Integrate with k6 / JMeter
- Reuse service layer

🔐 Security Extensibility

- Token validation
- Replay attack simulation
- Rate limiting checks

🧪 8. Example Implementation

📁 Sample Test

```
// tests/smoke/upiTransfer.spec.ts
import { test, expect } from '@playwright/test';
import { APIClient } from '../../client/apiClient';
import { UPIService } from '../../services/upi.service';

test('@smoke UPI transfer success', async () => {
  const client = new APIClient();
  await client.init(process.env.BASE_URL, process.env.TOKEN);

  const upi = new UPIService(client);

  const res = await upi.transfer({
    amount: 100,
    to: 'user@upi',
    requestId: Date.now()
  });

  expect(res.status).toBe(200);
  expect(res.body.status).toBe('SUCCESS');
});
```

✅ Custom Assertion Example

```
expect(res.body.transactionId).toBeTruthy();
```

🧠 Key Design Decisions

- **API-first architecture** → faster, stable
- **Service abstraction** → domain clarity
- **Strict assertions on money movement**
- **Idempotency baked into framework**
- **Data isolation → avoids flaky tests**
- **Lightweight smoke suite → fast CI gate**

✅ Final Takeaways

- Treat **every API call as financial risk**
- Validate **state, not just response**

- Prioritize:
  - Idempotency
  - Balance correctness
  - Limit enforcement
- Keep framework:
  - Modular
  - Observable
  - CI-ready

If you want next-level maturity, I can extend this into:

- 🏗️ Full repo scaffold (ready-to-run)
- 📊 Contract testing (OpenAPI validation)
- 🔄 Chaos testing for payment failures

End of task 2